《Ai Agent》第3-8节：动态实例化对话模型

来自： 码农会锁

小傅哥 2025年07月13日 16:23

本章重点： ★★★☆☆
课程视频： https://t.zsxq.com/zv79l
代码分支： https://gitcode.net/KnowledgePlanet/ai-agent-station-study/-/tree/3-8-bean-client-model
工程代码： https://gitcode.net/KnowledgePlanet/ai-agent-station-study

版权说明： ©本项目与星球签约合作，受《中华人民共和国著作权法实施条例》 版权法保护，禁止任何理由和任何方式公开(public)源码、资料、视频等小傅哥发布的星球内容到Github、Gitee等各类平台，违反可追究进一步的法律责任。

作者：小傅哥
博客：https://bugstack.cn

> 沉淀、分享、成长，让自己和他人都能有所收获！😄

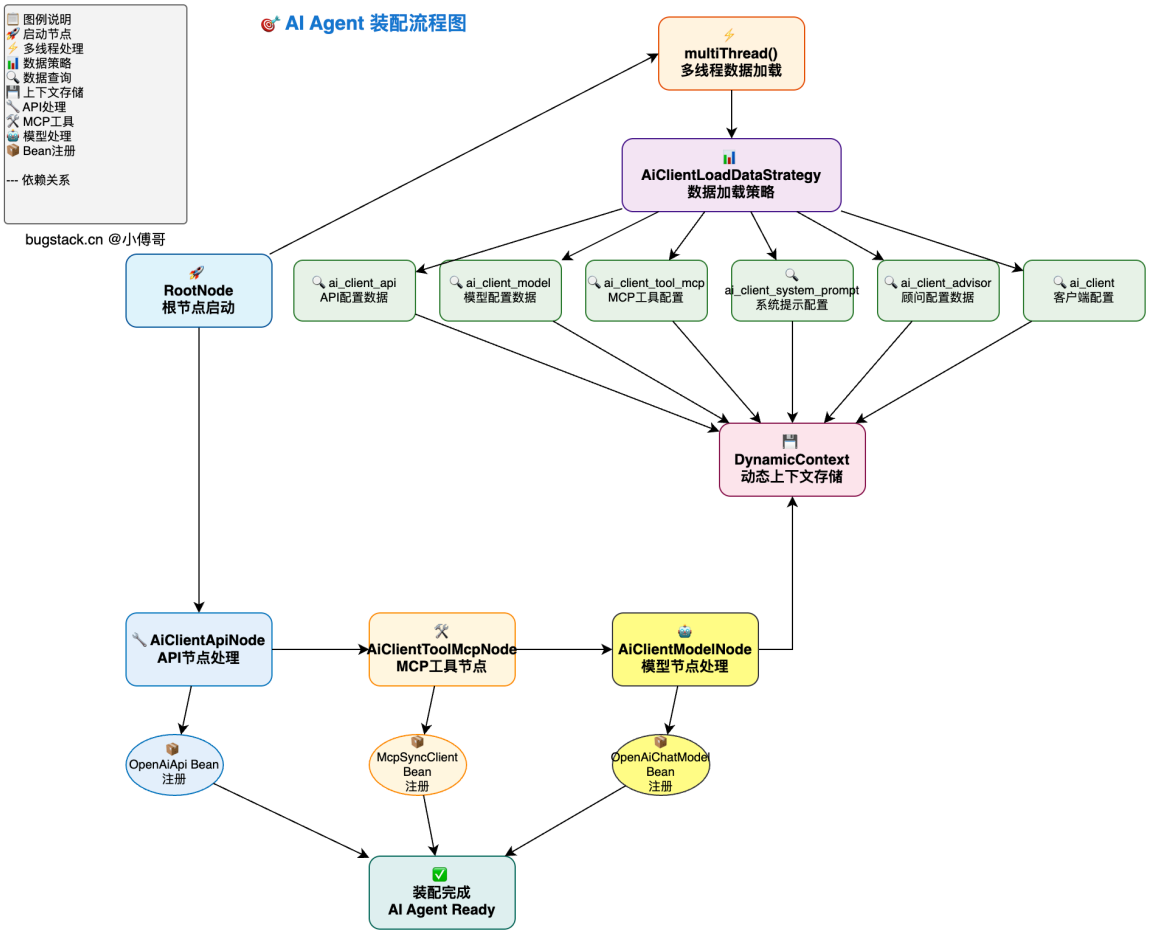
一、本章诉求
二、功能流程
三、编码实现
1. 工程结构
2. 修改说明
3. 核心改动
四、功能测试
1. 前置说明
2. 测试代码
五、读者作业

一、本章诉求

整个 ChatModel 对话模型，所需的元素包括；ai api、tool mcp 两块内容。那么本节我们就分别通过不同的 Node 节点实现这些元素的实例化和 ChatModel 构建。

二、功能流程

如图，ai api、tool mcp、model，实例化过程；

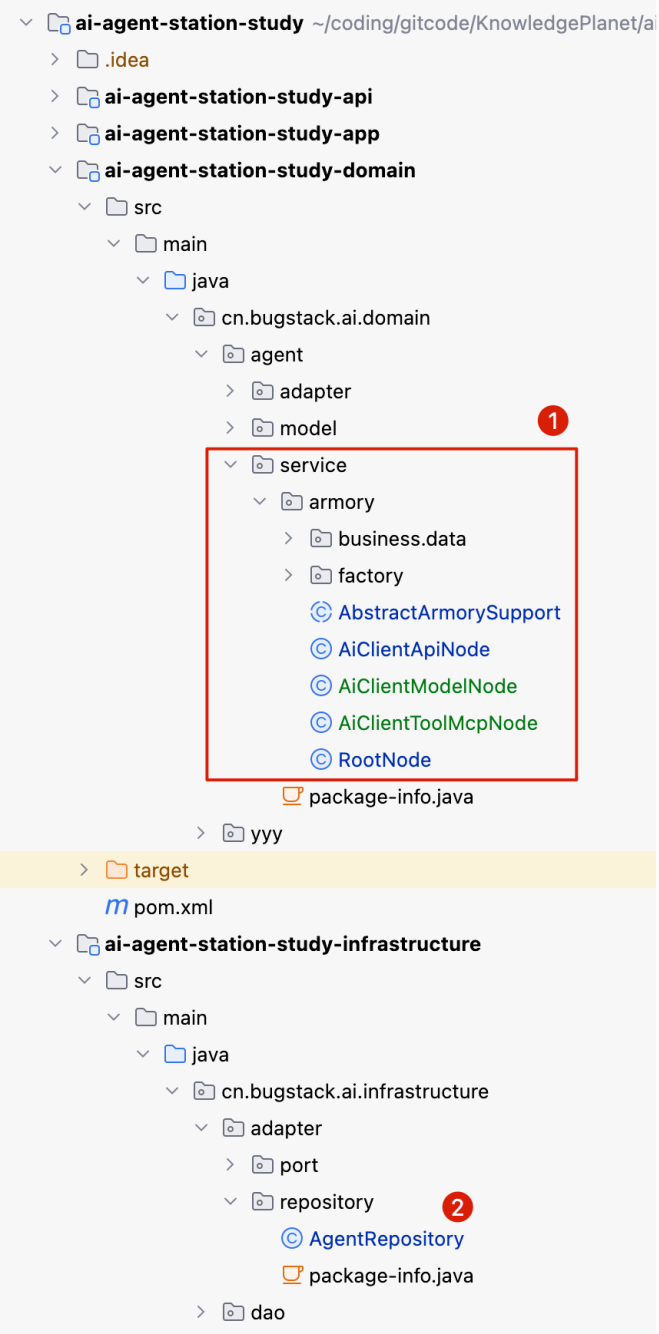


首先，如图 RootNode 负责数据加载，将构建节点元素的数据依次加载到内存中（写入到上下文里）

之后，除了上一节完成的 API 节点处理后，开始创建 MCP 服务的创建，之后是 ChatModel 对话模块的创建。因为 ChatModel 创建的时候会需要用到的 api、mcp 两个元素。

三、编码实现

1. 工程结构



1 构建服务

2 数据仓储

如图，绿色新增加了，AiClientToolMcpNode、AiClientModelNode，蓝色修改了 RootNode、AiClientApiNode 节点，以及抽象的装配职称类和数据仓库服务。

这些 XxxNode 节点，是一个循序调用的过程，从一个节点调用到下一个节点，顺序的把这些内容初始化完成。

2. 修改说明

1. AiClientModelVO 增加 toolMcpIds 信息，用于知道在构建 Model 对话对象时候，应该把哪些 MCP 加载进来。但要注意，这部分添加字段后，要在 AgentRepository#AiClientModelVOByClientIds 中，增加对 AiClientModel 的查询。
2. AgentRepository#AiClientToolMcpVOByClientIds 查询时，需要对 sse、stdio 配置，转换成 AiClientToolMcpVO 中对应的 transportConfigSse、transportConfigStdio 对象。
3. AiAgentEnumVO 枚举值，AI_CLIENT_TOOL_MCP code 调整为 tool_mcp 和数据库保持一致。
4. AiClientLoadDataStrategy 数据加载策略，AI_CLIENT_SYSTEM_PROMPT、AI_CLIENT_TOOL_MCP 两个配置反了，需要调整。
5. ArmoryCommandEntity 聚合进来一个 getLoadDataStrategy() 方法，替换 RootNode 中，说动编码的操作，通过命令，获取策略。让代码逻辑更加清晰。
6. 新增加 AiClientToolMcpNode，构建 MCP 服务。这部分需要根据，sse、stdio 两种 mcp 服务，分别实例化。
7. 新增加 AiClientModelNode，从 spring 容器中获取 ai api、tool mcp 实例化，关联到 model 上的元素，之后进行实例化操作。
8. 调整数据库表配置 ai_client_config，model 只配置有效关联 5003，其他的配置 status = 0 这部分会提供数据库语句，直接导入最新的即可
9. 调整数据库表配置 ai_client_api，api-key 字段需要配置有效值，如果你没有申请地方，可以找小傅哥获取渠道。同时对 application-dev.yml 配置文件，ai openai api-key 字段进行修改。

10. 对 AgentTest 测试类，增加 test_aiClientModelNode 以及验证创建的 OpenAiChatModel 进行对话。

> 以上为本节整体要修改的代码部分，基本你可以参考这些内容尝试编码。往往这也是公司里，研发在做设计阶段，需要列举出的。

3. 核心改动

3.1 AiClientApiNode

```
@Service
public class AiClientApiNode extends AbstractArmorySupport {

    @Resource
    private AiClientToolMcpNode aiClientToolMcpNode;

    @Override
    protected String doApply(ArmoryCommandEntity requestParameter, DefaultArmoryStrategyFactory.Dy
        log.info("Ai Agent 构建节点，API 接口请求{}", JSON.toJSONString(requestParameter));

        List<AiClientApiVO> aiClientApiList = dynamicContext.getValue(dataName());

        if (aiClientApiList == null || aiClientApiList.isEmpty()) {
            log.warn("没有需要被初始化的 ai client api");
            return router(requestParameter, dynamicContext);
        }

        for (AiClientApiVO aiClientApiVO : aiClientApiList) {
            // 构建 OpenAiApi
            OpenAiApi openAiApi = OpenAiApi.builder()
                    .baseUrl(aiClientApiVO.getBaseUrl())
                    .apiKey(aiClientApiVO.getApiKey())
                    .completionsPath(aiClientApiVO.getCompletionsPath())
                    .embeddingsPath(aiClientApiVO.getEmbeddingsPath())
                    .build();

            // 注册 OpenAiApi Bean 对象
            registerBean(beanName(aiClientApiVO.getApiId()), OpenAiApi.class, openAiApi);
        }

        return router(requestParameter, dynamicContext);
    }

    @Override
    public StrategyHandler<ArmoryCommandEntity, DefaultArmoryStrategyFactory.DynamicContext, Strin
        return aiClientToolMcpNode;
    }

    @Override
    protected String beanName(String beanId) {
        return AiAgentEnumVO.AI_CLIENT_API.getBeanName(beanId);
    }

    @Override
    protected String dataName() {
        return AiAgentEnumVO.AI_CLIENT_API.getDataName();
    }

}
```

首先，在 AbstractArmorySupport 定义 beanName、dataName，让每个节点类都可以实现一个自己的 beanName、dataName 方法，这样更好维护。

之后，在 `dynamicContext.getValue(dataName());` 使用 dataName 方法，获取 aiClientApiList 数据。

最后，AiClientApiNode 节点完成后，要路由到下一个节点，aiClientToolMcpNode。也就是 get 方法里新增加的配置。

3.2 AiClientToolMcpNode

```
@Service
public class AiClientToolMcpNode extends AbstractArmorySupport {

    @Resource
    private AiClientModelNode aiClientModelNode;

    @Override
    protected String doApply(ArmoryCommandEntity requestParameter, DefaultArmoryStrategyFactory.Dy
        log.info("Ai Agent 构建节点，Tool MCP 工具配置{}", JSON.toJSONString(requestParameter));

        List<AiClientToolMcpVO> aiClientToolMcpList = dynamicContext.getValue(dataName());

        if (aiClientToolMcpList == null || aiClientToolMcpList.isEmpty()) {
            log.warn("没有需要被初始化的 ai client tool mcp");
            return router(requestParameter, dynamicContext);
        }

        for (AiClientToolMcpVO mcpVO : aiClientToolMcpList) {
            // 创建 MCP 服务
            McpSyncClient mcpSyncClient = createMcpSyncClient(mcpVO);

            // 注册 MCP 对象
            registerBean(beanName(mcpVO.getMcpId()), McpSyncClient.class, mcpSyncClient);
        }

        return router(requestParameter, dynamicContext);
    }

    @Override
    public StrategyHandler<ArmoryCommandEntity, DefaultArmoryStrategyFactory.DynamicContext, Strin
        return aiClientModelNode;
    }

    @Override
    protected String beanName(String beanId) {
        return AiAgentEnumVO.AI_CLIENT_TOOL_MCP.getBeanName(beanId);
    }

    @Override
    protected String dataName() {
        return AiAgentEnumVO.AI_CLIENT_TOOL_MCP.getDataName();
    }

    private McpSyncClient createMcpSyncClient(AiClientToolMcpVO aiClientToolMcpVO) {
        String transportType = aiClientToolMcpVO.getTransportType();

        switch (transportType) {
            case "sse" -> {
                AiClientToolMcpVO.TransportConfigSse transportConfigSse = aiClientToolMcpVO.getTra
                // http://127.0.0.1:9999/sse?apikey=DElk89iu8Ehhnbu
                String originalBaseUri = transportConfigSse.getBaseUri();
                String baseUri;
                String sseEndpoint;

                int queryParamStartIndex = originalBaseUri.indexOf("sse");
                if (queryParamStartIndex != -1) {
                    baseUri = originalBaseUri.substring(0, queryParamStartIndex - 1);
                    sseEndpoint = originalBaseUri.substring(queryParamStartIndex - 1);
                } else {
                    baseUri = originalBaseUri;
```

```
                    sseEndpoint = transportConfigSse.getSseEndpoint();
                }

                sseEndpoint = StringUtils.isBlank(sseEndpoint) ? "/sse" : sseEndpoint;

                HttpClientSseClientTransport sseClientTransport = HttpClientSseClientTransport
                        .builder(baseUri) // 使用截取后的 baseUri
                        .sseEndpoint(sseEndpoint) // 使用截取或默认的 sseEndpoint
                        .build();

                McpSyncClient mcpSyncClient = McpClient.sync(sseClientTransport).requestTimeout(Du
                var init_sse = mcpSyncClient.initialize();

                log.info("Tool SSE MCP Initialized {}", init_sse);
                return mcpSyncClient;
            }
            case "stdio" -> {
                AiClientToolMcpVO.TransportConfigStdio transportConfigStdio = aiClientToolMcpVO.ge
                Map<String, AiClientToolMcpVO.TransportConfigStdio.Stdio> stdioMap = transportConf
                AiClientToolMcpVO.TransportConfigStdio.Stdio stdio = stdioMap.get(aiClientToolMcpV

                // https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem
                var stdioParams = ServerParameters.builder(stdio.getCommand())
                        .args(stdio.getArgs())
                        .env(stdio.getEnv())
                        .build();

                var mcpClient = McpClient.sync(new StdioClientTransport(stdioParams))
                        .requestTimeout(Duration.ofSeconds(aiClientToolMcpVO.getRequestTimeout()))
                var init_stdio = mcpClient.initialize();

                log.info("Tool Stdio MCP Initialized {}", init_stdio);
                return mcpClient;
            }
        }

        throw new RuntimeException("err! transportType " + transportType + " not exist!");
    }

}
```

```
public McpSyncClient stdioMcpClient() {
    // based on
    // https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem
    var stdioParams = ServerParameters.builder("npx")
            .args("-y", "@modelcontextprotocol/server-filesystem", "/Users/fuzhengwei/Desktop", "/
            .build();
    var mcpClient = McpClient.sync(new StdioClientTransport(stdioParams))
            .requestTimeout(Duration.ofSeconds(10)).build();
    var init = mcpClient.initialize();
    System.out.println("Stdio MCP Initialized: " + init);
    return mcpClient;
}

public McpSyncClient sseMcpClient01() {
    HttpClientSseClientTransport sseClientTransport = HttpClientSseClientTransport.builder("http:/
    McpSyncClient mcpSyncClient = McpClient.sync(sseClientTransport).requestTimeout(Duration.ofMin
    var init = mcpSyncClient.initialize();
    System.out.println("SSE MCP Initialized: " + init);
    return mcpSyncClient;
```

```
}
```

首先，我们看下，在 stdioMcpClient、sseMcpClient01 分别有对应的编码实现，本节的操作就是把部分手动创建的过程，放到 AiClientToolMcpNode 中实现。

之后，createMcpSyncClient 是这个节点抽取的核心方法，根据 sse、stdio 两种方式分别创建。

最后，doApply 调用 createMcpSyncClient 方法，并把创建的对象，注册到 Spring 容器中。

3.3 AiClientModelNode

```
@Service
public class AiClientModelNode extends AbstractArmorySupport {

    @Override
    protected String doApply(ArmoryCommandEntity requestParameter, DefaultArmoryStrategyFactory.Dy
        log.info("Ai Agent 构建节点，Mode 对话模型{}", JSON.toJSONString(requestParameter));

        List<AiClientModelVO> aiClientModelList = dynamicContext.getValue(dataName());

        if (aiClientModelList == null || aiClientModelList.isEmpty()) {
            log.warn("没有需要被初始化的 ai client model");
            return router(requestParameter, dynamicContext);
        }

        for (AiClientModelVO modelVO : aiClientModelList) {

            // 获取当前模型关联的 API Bean 对象
            OpenAiApi openAiApi = getBean(AiAgentEnumVO.AI_CLIENT_API.getBeanName(modelVO.getApiId
            if (null == openAiApi) {
                throw new RuntimeException("mode 2 api is null");
            }

            // 获取当前模型关联的 Tool MCP Bean 对象
            List<McpSyncClient> mcpSyncClients = new ArrayList<>();
            for (String toolMcpId : modelVO.getToolMcpIds()) {
                McpSyncClient mcpSyncClient = getBean(AiAgentEnumVO.AI_CLIENT_TOOL_MCP.getBeanName
                mcpSyncClients.add(mcpSyncClient);
            }

            // 实例化对话模型（如果有其他模型对接，可以使用 one-api 服务，转换为 openai 模型格式）
            OpenAiChatModel chatModel = OpenAiChatModel.builder()
                    .openAiApi(openAiApi)
                    .defaultOptions(
                            OpenAiChatOptions.builder()
                                    .model(modelVO.getModelName())
                                    .toolCallbacks(new SyncMcpToolCallbackProvider(mcpSyncClients)
                                    .build())
                    .build();

            // 注册 Bean 对象
            registerBean(beanName(modelVO.getModelId()), OpenAiChatModel.class, chatModel);
        }

        return router(requestParameter, dynamicContext);
    }

    @Override
    public StrategyHandler<ArmoryCommandEntity, DefaultArmoryStrategyFactory.DynamicContext, Strin
        return defaultStrategyHandler;
    }

    @Override
```

```
    protected String beanName(String beanId) {
        return AiAgentEnumVO.AI_CLIENT_MODEL.getBeanName(beanId);
    }

    @Override
    protected String dataName() {
        return AiAgentEnumVO.AI_CLIENT_MODEL.getDataName();
    }

}
```

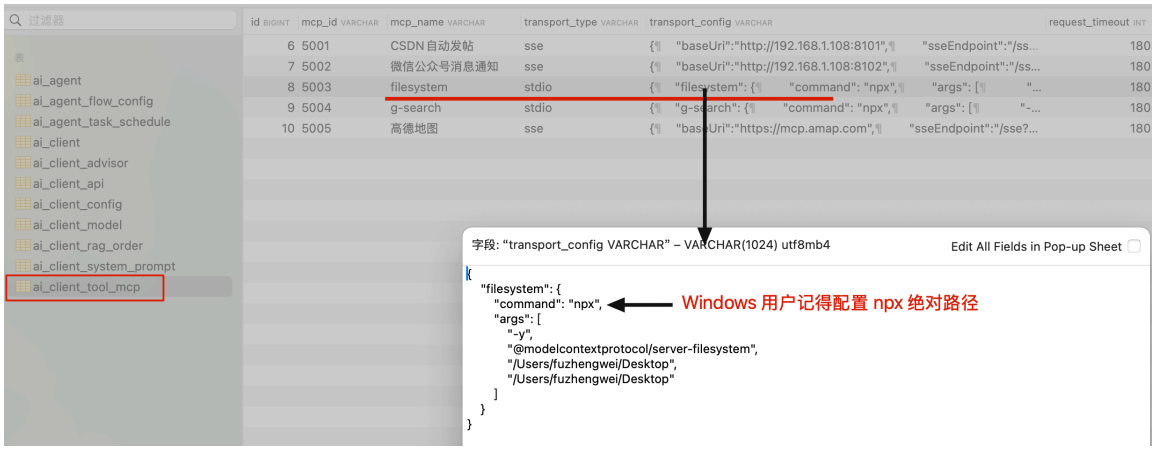
这个节点的作用，就是构建出 ChatMode 对话模型。他所需的 OpenAiApi、McpSyncClient 都是通过 Spring 容器进行获取的。

构建完成后，将 ChatMode 注册到 Spring 容器即可。

> 其他一些为了支撑核心类的代码，可以直接阅读源码，也可以看课程视频，会详细讲解，以及带着完成。

四、功能测试

1. 前置说明



注意，导入的库表数据，ai_client_tool_mcp filesystem npx Windows 用户需要修改。Mac、linux 可以直接使用。

这些内容在前面 rag、mcp 部分都有讲解，可以直接对照来看。

2. 测试代码

```
@Test
public void test_aiClientModelNode() throws Exception {
    StrategyHandler<ArmoryCommandEntity, DefaultArmoryStrategyFactory.DynamicContext, String> armo
            defaultArmoryStrategyFactory.armoryStrategyHandler();

    String apply = armoryStrategyHandler.apply(
            ArmoryCommandEntity.builder()
                    .commandType(AiAgentEnumVO.AI_CLIENT.getCode())
                    .commandIdList(Arrays.asList("3001"))
                    .build(),
            new DefaultArmoryStrategyFactory.DynamicContext());

    OpenAiChatModel openAiChatModel = (OpenAiChatModel) applicationContext.getBean(AiAgentEnumVO.A
    log.info("模型构建:{}", openAiChatModel);

    // 1. 有哪些工具可以使用
    // 2. 在 /Users/fuzhengwei/Desktop 创建 txt.md 文件
    Prompt prompt = Prompt.builder()
            .messages(new UserMessage(
                    """
                            有哪些工具可以使用
```

```
                        """))
            .build();

    ChatResponse chatResponse = openAiChatModel.call(prompt);

    log.info("测试结果(call):{}", JSON.toJSONString(chatResponse));
}
```

```
25-07-05.16:45:58.784 [pool-2-thread-1 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client_api
25-07-05.16:45:58.784 [pool-2-thread-2 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client_mod
25-07-05.16:45:58.784 [pool-2-thread-4 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client_sys
25-07-05.16:45:58.784 [pool-2-thread-3 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client_too
25-07-05.16:45:58.784 [pool-2-thread-5 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client_adv
25-07-05.16:45:58.784 [pool-2-thread-6 ] INFO  AiClientLoadDataStrategy - 查询配置数据(ai_client) [3
25-07-05.16:45:58.791 [pool-2-thread-6 ] INFO  HikariDataSource       - MainHikariPool - Starting.
25-07-05.16:45:58.862 [pool-2-thread-6 ] INFO  HikariPool             - MainHikariPool - Added con
25-07-05.16:45:58.863 [pool-2-thread-6 ] INFO  HikariDataSource       - MainHikariPool - Start com
25-07-05.16:45:58.935 [main            ] INFO  RootNode               - Ai Agent 构建，数据加载节点{"
25-07-05.16:45:58.936 [main            ] INFO  AiClientApiNode        - Ai Agent 构建节点，API 接口请
25-07-05.16:45:58.936 [main            ] INFO  AbstractArmorySupport  - 成功注册Bean: ai_client_api_
25-07-05.16:45:58.936 [main            ] INFO  AiClientToolMcpNode    - Ai Agent 构建节点，Tool MCP
25-07-05.16:46:00.543 [pool-6-thread-1 ] INFO  StdioClientTransport   - STDERR Message received: S
25-07-05.16:46:00.544 [pool-6-thread-1 ] INFO  StdioClientTransport   - STDERR Message received: A
25-07-05.16:46:00.562 [pool-3-thread-1 ] INFO  McpAsyncClient         - Server response with Proto
25-07-05.16:46:00.565 [main            ] INFO  AiClientToolMcpNode    - Tool Stdio MCP Initialized
25-07-05.16:46:00.566 [main            ] INFO  AbstractArmorySupport  - 成功注册Bean: ai_client_too
25-07-05.16:46:00.566 [main            ] INFO  AiClientModelNode      - Ai Agent 构建节点，Mode 对话
25-07-05.16:46:00.576 [main            ] INFO  AbstractArmorySupport  - 成功注册Bean: ai_client_mod
25-07-05.16:46:00.576 [main            ] INFO  AgentTest              - 模型构建:OpenAiChatModel [d
25-07-05.16:46:05.196 [main            ] INFO  AgentTest              - 测试结果(call):{"metadata":
```

3001 客户端 AI ID，2001 ChatModel 对话模型ID，一个是根据 3001 客户端 ID 加载，另外一个 2001 是根据 ChatMode 对话模型 ID 获取对象。

当获取了 OpenAiChatModel 对象后，就可以发起对话了，这里我们可以询问 有哪些工具可以使用，也可以询问 在 `/Users/fuzhengwei/Desktop` 创建 `txt.md` 文件

五、读者作业

简单作业：完成本节内容，实例化 Tool Mcp、Model 对话模型。完成测试验证。

复杂作业：尝试完成顾问角色，提示词节点的创建，并测试验证获取对象。

知识星球